CSS311 – Parallel & Distributed Computing

Programming Assignment — 2

Part A — MPI Implementation (Version 5 and Version 6)

Using the same problem, input format, and correctness check (checksum / tolerance) established in PA1, implement two additional versions:

Version 5 — Basic MPI Implementation

- Decompose the input data across MPI processes (domain decomposition — split the array, matrix, or dataset into contiguous or block chunks, one per rank).
- Use point-to-point and/or basic collective communication (`MPI_Bcast`, `MPI_Scatter`, `MPI_Gather`, `MPI_Reduce`) to distribute input and collect results.
- Must produce results identical to the Sequential, OpenMP, and CUDA versions.

Version 6 — Optimized MPI Implementation

Improve Version 5 using at least one of the following MPI concepts, chosen based on which is most relevant to your specific problem. Justify the choice in the report.

- **Collective operations:** replace manual point-to-point exchanges with the appropriate collective (`MPI_Allreduce`, `MPI_Allgather`, `MPI_Alltoall`, etc.) where it reduces communication steps or code complexity.
- **Non-blocking collectives:** use `MPI_Ireduce`, `MPI_Iallgather`, `MPI_Ibcast`, etc. to overlap communication with local computation, and measure the overlap achieved.
- **Process topologies:** for 2-D grid/image/matrix problems, create a Cartesian topology with `MPI_Cart_create` and implement halo/ghost-cell exchange with neighbouring ranks (`MPI_Cart_shift + Sendrecv or non-blocking sends/receives`).
- **Parallel I/O:** use MPI-IO (`MPI_File_open`, `MPI_File_read_at / write_at`, or `MPI_File_set_view`) so every rank reads its portion of a large input file directly, instead of one rank reading and scattering.
- **Single-sided communication:** use `MPI_Win_create` together with `MPI_Put / MPI_Get` (and `MPI_Win_fence or passive-target synchronization`) for problems with irregular or data-dependent access patterns where two-sided messaging adds unnecessary synchronization.

Optimized version is not restricted to the five concepts above. Any other MPI-relevant optimization (e.g. *custom derived datatypes, communicator splitting, process-to-processor mapping/rank placement, load-balanced or irregular decomposition, hybrid MPI+OpenMP*, etc.) may be used instead or in addition, provided it is properly justified.

Required Analysis for Part A (consider all these points while preparing the report)

- Identify how the domain decomposition maps to MPI ranks and whether it is block, cyclic, or block-cyclic, and why.
- Identify all communication points: *what is exchanged, between which ranks, and how often*.
- State whether communication is a bottleneck at small vs. large process counts, and explain using Amdahl/communication-overhead reasoning.
- Explain the specific MPI optimization used in Version 6 and why it should help this specific problem.
- Test with at least 3 process counts (e.g. 2, 8, 16) at each of the 3 input sizes used in PA1 (Small / Medium / Large).

Part B — Performance Analysis Using Tools

All versions including Sequential, OpenMP, CUDA, Optimized (from PA1), and MPI, MPI-Optimized (from Part A) must be examined using the following three categories of performance tools.

B.1 Profiling (CPU and GPU versions)

Applies to: Sequential, OpenMP, CUDA, Optimized.

- **CPU versions:** use `gprof or perf (perf stat, perf record + perf report)` to identify hotspot functions, cache-miss rates, and CPU cycles consumed by each phase of the computation.
- **GPU versions:** use `NVIDIA Nsight Compute` (or `nvprof` on older toolkits) to report kernel execution time, achieved occupancy, memory throughput (global load/store efficiency), and whether the kernel is compute-bound or memory-bound.
- Report, in a table, the top 2–3 hotspots per version and what fraction of total runtime each account for.

B.2 MPI Profiling

Applies to: MPI, MPI-Optimized.

- **Tool:** `mpiP` — a lightweight, low-overhead MPI profiling library. Link it against your MPI binary (no source changes required) and run normally; it produces a text report on program exit.
- Report, per MPI version: total time spent in MPI calls vs. computation time, a breakdown by MPI call type (`Send/Recv/Bcast/Reduce`/etc.), and which call dominates communication overhead.
- Compare this breakdown across process counts (e.g. 2 vs. 16 ranks) and explain how communication overhead scales.

B.3 Energy Analysis

Applies to: all versions — Sequential, OpenMP, CUDA, Optimized, MPI, MPI-Optimized.

- **CPU energy:** Intel RAPL counters via `perf stat -e power/energy-pkg/,power/energy-ram/` (or an equivalent RAPL-reading tool available on your lab machines).
- **GPU energy:** `nvidia-smi --query-gpu=power.draw --format=csv -l 1` sampled during kernel execution, or Nsight Compute's power metrics if available.
- Report `energy consumed (joules)` for each version at each input size, and derive an efficiency metric: performance-per-watt or results-per-joule.

B.4 If a Tool Is Not Supported on Your Device

The table below gives the accepted fallback for each tool. Use the fallback only after a genuine attempt to get the primary tool working. Clearly state in your report what was tried and why it did not work.

If this tool is unavailable…	Primary Requirement	Use this fallback instead
perf (CPU profiling)	B.1 — CPU hotspots	gprof (compile with -pg, run, then gprof <binary> gmon.out) for function-level hotspots and call counts.
Nsight Compute / nvprof (GPU profiling)	B.1 — GPU kernel metrics	CUDA event timing (cudaEventCreate / cudaEventRecord / cudaEventElapsedTime) around the kernel launch for kernel time, plus effective bandwidth = (bytes read + bytes written) / kernel time.
mpiP (MPI profiling)	B.2 — MPI communication breakdown	Manual instrumentation with MPI_Wtime() around your computation block and around each communication call, accumulated into a

If this tool is unavailable…	Primary Requirement	Use this fallback instead
		computation-time vs. communication-time total per rank.
RAPL (CPU energy)	B.3 — CPU energy	Report wall-clock time only for CPU-only versions and state in the report that CPU energy could not be measured on the available hardware.
nvidia-smi power query (GPU energy)	B.3 — GPU energy	If nvidia-smi power draw is also unavailable, report GPU energy as "not measurable on this device" and rely on kernel time and bandwidth (from B.1) as the GPU performance evidence instead.

Part C — Final Consolidated Comparison (All Versions, All Datasets)

After completing part B, using the results and tool outputs from PA1 and PA2, produce a single consolidated comparison covering all versions of your problem:

Version	Small Time (s)	Medium Time (s)	Large Time (s)	Speedup (vs Seq.)	Parallel Efficiency	Energy (J)	Top Bottleneck	Directly Comparable To
Sequential								
OpenMP								
CUDA								
Optimized (PA1)								
MPI (basic)								
MPI (optimized)								

Comparability Discussion (Required)

Not every version can be compared on equal footing. In your report, explicitly address:

- **Same-resource comparisons:** Sequential vs. OpenMP vs. MPI-on-one-node use the same CPU hardware and are directly comparable on wall-clock time and speedup.
- **Cross-hardware comparisons:** CUDA (GPU) vs. CPU versions are not directly comparable on raw time alone. Discuss using total end-to-end time (including host–device transfer) versus kernel-only time, and note that speedup numbers depend heavily on which one is reported.
- **Scaling comparisons:** MPI process count vs. OpenMP thread count are not the same resource unit. Compare parallel efficiency (speedup / resource count) rather than raw speedup when contrasting MPI and OpenMP scaling behaviour.
- **Energy comparisons:** CPU-only and GPU-involving versions draw power from different components (package vs. board). State clearly what each energy number includes before comparing efficiency across versions.

Conclude the report with a short-written summary (≤ 1 page). State which version was fastest overall, which was most energy-efficient, which had the best parallel efficiency, and whether the fastest version was also the most resource-efficient. Explain any mismatch.

Expected Deliverables

- Source code for Version 5 (MPI — basic) and Version 6 (MPI — optimized).
- Profiling reports (gprof/perf output for CPU versions; Nsight Compute/nvprof output for CUDA) for all four PA1 versions.
- mpiP reports for both MPI versions, at all tested process counts.
- Energy measurement logs (RAPL/nvidia-smi) for all six versions.
- The completed consolidated comparison table (Part C) plus the comparability discussion and summary.
- An updated report section covering the Part A required analysis (decomposition, communication points, bottleneck discussion, optimization justification).
- All group members must be able to explain the MPI implementation, the profiling results, and the final comparison during evaluation.

Note: *Where any input, parameter, threshold, or other detail is not specified, students may make reasonable assumptions and use suitable sample, synthetic, or random data. Clearly state the assumptions made and use the same logical input and assumptions as PA1 across all implementations. Additional steps or parameters may be introduced if required to complete the assigned problem, provided they are relevant and justified.*